
**РАБОЧИЙ ПРОТОКОЛ И ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3 "[C++
и UNIX]: C++ CLI / FUNCTION / LOOP / RECURSION"**

Группа: Z3344
Студенты: Поздеев Дмитрий
Преподаватель: Маслов И.

К работе допущен:
Работа выполнена:
Отчет принят:

1 Цель работы

- Познакомить студента с основными алгоритмическими конструкциями, которые будут использоваться для создания CLI программы. Далее продемонстрировать эффективность использования механизма рекурсии.
- C++ алгоритмы: CLI Калькулятор вещественных чисел +, -, **, . Реализация с использованием только функций, условий, циклов, + и -.
- Вид команд в консоли: calc plus / minus / power; Ханойская башня, результат корректной последовательности

2 Задачи, решаемые при выполнении работы

- C++ CLI CALC Создать программу CALC с интерфейсом CLI
- C++ RECURSION Решить задачу ханойской башни с использованием рекурсии
- SAVE Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

3 Код

3.1 Блок с CLI

Листинг 1: Код на плюсах

```
#include <iostream>
#include <string>
#include <cmath>

using namespace std;

double power(double base, int exponent) {
    if (exponent == 0) {
        if (base == 0) {
            throw runtime_error("0^0_is_undefined.");
        }
        return 1;
    }
    if (base == 0 && exponent < 0) {
        throw runtime_error("Division_by_zero_(0_raised_to_a_negative_power).");
    }
    bool negative = exponent < 0;
    exponent = abs(exponent);

    double result = 1;
    for (int i = 0; i < exponent; ++i) {
        result *= base;
    }
    return negative ? 1.0 / result : result;
}

int main() {
    double a, b;
    string op;

    //
    cout << "Enter_first_number:_";
    cin >> a;

    //
    cout << "Enter_operator_(plus/minus/power):_";
    cin >> op;

    //
    cout << "Enter_second_number:_";
    cin >> b;

    try {
        if (op == "plus") {
            cout << "Result:_ " << a + b << endl;
        } else if (op == "minus") {
            cout << "Result:_ " << a - b << endl;
        } else if (op == "power") {
            if (floor(b) != b) {

```

```

        throw runtime_error("Exponent_must_be_an_integer.");
    }
    int exponent = static_cast<int>(b);
    cout << "Result:_" << power(a, exponent) << endl;
} else {
    throw runtime_error("Invalid_operator.");
}
} catch (const exception& e) {
    cerr << "Error:_" << e.what() << endl;
    return 1;
}

return 0;
}

```

3.2 Блок про ханойскую башню

Листинг 2: код на плюсах

```

import time

#include <iostream>
#include <string>

using namespace std;

struct ListNode {
    string move;
    ListNode* prev;
    ListNode* next;
    ListNode(const string& m) : move(m), prev(nullptr), next(nullptr) {}
};

class DoublyLinkedList {
public:
    ListNode* head;
    ListNode* tail;

    DoublyLinkedList() : head(nullptr), tail(nullptr) {}

    void append(const string& move) {
        ListNode* newNode = new ListNode(move);
        if (!head) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
}

```

```

void print() const {
    ListNode* current = head;
    while (current) {
        cout << current->move << endl;
        current = current->next;
    }
}

~DoublyLinkedList() {
    ListNode* current = head;
    while (current) {
        ListNode* next = current->next;
        delete current;
        current = next;
    }
}

};

void hanoi(int n, char from, char to, char aux, DoublyLinkedList& list) {
    if (n == 1) {
        list.append("Move_disk_1_from_" + string(1, from) + "_to_" + string(1, to));
        return;
    }
    hanoi(n - 1, from, aux, to, list);
    list.append("Move_disk_" + to_string(n) + "_from_" + string(1, from) + "_to_" + string(1, to));
    hanoi(n - 1, aux, to, from, list);
}

int main() {
    const int disks = 8;
    DoublyLinkedList list;
    hanoi(disks, 'A', 'C', 'B', list);
    list.print();
    return 0;
}

```

3.3 Работа с Git

Листинг 3: Клонирование репозитория и создание структуры

```

git clone https://github.com/PozdeevDA/my_labs.git
cd my_labs
mkdir -p lab_01/{build,src,doc,cmake} lab_02/{build,src,doc,cmake}
git checkout -b dev
git push -u origin dev
git branch -d main
git push origin --delete main

```

Листинг 4: Скрипт для переноса из dev в stg:

```
git add .  
git commit -m "last_fix"  
git push origin dev
```

4 Выводы

В ходе работы освоены базовые алгоритмические конструкции C++ через реализацию CLI-калькулятора с использованием функций, циклов и условий. Рекурсивное решение задачи Ханойской башни продемонстрировало эффективность рекурсии и навыки работы с динамическими структурами данных.